

Quarto Manuals

Tinashe M. Tapera¹, Chris¹, Danielle^{2,3}, and Michelle⁴

¹Department of Psychology, Ana and Blanca's University

²Danielle's Primary Affiliation

³Danielle's Secondary Affiliation

⁴Buffalo, NY

Author Note

Tinashe M. Tapera  <https://orcid.org/0000-0001-9080-5010>

Chris  <https://orcid.org/0000-0000-0000-0002>

Danielle  <https://orcid.org/0000-0000-0000-0003>

Michelle  <https://orcid.org/0000-0000-0000-0004>

Carina Mengano is now at Generic University.

The authors have no conflicts of interest to disclose. Author roles were classified using the Contributor Role Taxonomy (CRediT; credit.niso.org) as follows: Tinashe M. Tapera: conceptualization, writing; Chris: project administration, formal analysis; Danielle: formal analysis, writing; Michelle: writing, methodology, formal analysis

Correspondence concerning this article should be addressed to Tinashe M. Tapera, Department of Psychology, Ana and Blanca's University, 1234 Capital St., Albany, NY 12084-1234, USA, Email: ttapera@hsph.harvard.edu

Abstract

to be fleshed out from bullet form below

Impact Statement

Scientific programming workflows are becoming increasingly complex, while existing support mechanisms remain poorly matched to that complexity. Traditional documentation is flexible but non-executable, command-line wrappers are executable but often too rigid, and current agentic systems are promising but not yet sufficiently reliable for many high-stakes workflow setup tasks. We propose Quarto Manuals, an executable documentation framework built on Quarto that helps authors create flexible, scalable, and reproducible workflow manuals composed of structured pages with explicit procedures and checks. Powered by the quarto-emit backend, Quarto Manuals enable authors to create executable documentation that allows users to conveniently step through scientific computing workflows while generating reproducible project artifacts and receiving immediate feedback on whether each step has been implemented correctly.

Keywords: APA Style, Quarto, Markdown, apaquarto

Quarto Manuals

Abstract

- programming is already hard and the data is only going to get bigger. workflows are blowing up in complexity and scale. scientists need rigour in their workflows.
- to get around this, we could write better and better docs. but even great docs have limitations:
 - docs are something you *read*, not something you *do*, meaning they break down in usefulness when we have to scale to longer and more complicated procedures
 - docs are not executable, meaning artifacts of progress are not reproducible, testable, or verifiable
 - good docs do not have a flexible feedback loop to confirm the user is acting correctly
- *What about building command line wrappers around existing tools?* They are executable but they are totally inflexible.
 - You build it once and everyone has to do it that way, or
 - You inundate your software with several flags, arguments, and runtime options to try to make it flexible. This is not a good solution for repetitive but unexpectedly parametric workflows.
- *What about AI, LLMs, agents?*
 - This is a promising solution, but (fact) LLMs are non-deterministic, meaning some steps in larger and more complex workflows will inevitably have results that may not be reproducible.
 - Additionally (opinion) agents are not yet mature enough to be reliable for some of the complex workflows we have in mind (though we are working on it with agentic

workflows). Most users are still in the dyadic, “paste text into the box and read what comes out” turn-based workflow, which is not sufficient for many of the complex workflows we do.

- To solve this, we need a new type of documentation that is executable, flexible, and scalable. This is where Quarto and notebook-driven-development (NDD) comes in.
- We propose Quarto Manuals, an executable documentation framework that is flexible, scalable, and reproducible. Internally, a Quarto Manual is a custom starter template and extension for Quarto.
- benefits:
 - flexible: can be used for different audiences, can be used for different purposes, can be used for different programming languages
 - scalable: can be used for different projects, can be used for different teams, can be used for different organizations, and can scale in complexity required of the project
 - reproducible: side effects are intentional, meaning they can be used to generate reproducible artifacts for the workflow itself
 - relies on quarto and can leverage the full ecosystem of quarto, including quarto extensions, quarto publishing, and quarto rendering
- drawbacks:
 - requires some programming knowledge to use
 - requires quarto; might be a new framework for some
 - requires active user participation, not fully automated (but this is a feature, not a bug)
- we show three examples in action: a simple example, an intermediate example, and a complex example.

- Importantly, we use `quarto-emit`, a new extension, for the backend. A Quarto Manual is just a cognitive framework physically materialized by Quarto’s custom Divs, and powered in the backend by a simple but powerful file export extension, `quarto-emit`.
- Workflow for manual *authors*:
 - Authors call `quarto use template`
`GoldenPlanetaryHealthLab/QuartoManuals` to create a new Quarto Manual that instructs users on how to do something complex and procedural, for e.g.,
“SetupMyProject”
 - Helpful prompts throughout the template help guide manual authors to use the template effectively, but leave them with the flexibility to implement their workflows to their desired level of sophistication.
 - Authors implement each page check with a testing suite to give the user feedback on whether they are implementing the workflow correctly.
 - inter-page dependencies are gently suggested, but not enforced to reduce implementation overhead for the extension and allow for flexibility in workflow design.
 - Authors `git push` their manuals to an organization’s GitHub repository (or code hosting site of your choice)
- For manual *users*:
 - In your home directory or otherwise, `quarto use template`
`MyLab/SetupMyProject` clones the SetupMyProject Manual to your local machine.
 - Step through the pages of the manual, hardcoding critical, context-specific variables (like your cluster configuration, your desired Python version, etc.) and executing the code as you go.

- With each page, leave behind setup artifacts as you go, like environment files (`.venv`, `.Renv`, `.Renviron`) scripts (`setup.sh`, `install.R`), and other configuration files (`config.toml`, `settings.json`, `pyproject.toml`).
- Run `quarto render` to render the whole manual to HTML, PDF, or DOCX. If the manual has checks implemented in a testing suite, `quarto render` will execute the testing suite, providing you with a visual report of your progress in the manual and whether you are implementing the workflow correctly.
- If all goes well, you now have a fully setup project as the original author intended, and you can now start working on your implementation of the project. If it doesn't you can go back to the exact Point of Failure (POF) and fix the issue, or you can reach out to the original author for help.

Hypotheses, Aims, and Objectives

The last paragraph of the introduction usually states the specific hypotheses of the study, often in a way that links them to the research design.

Method

General remarks on method. This paragraph is optional. Not all papers require each of these sections. Edit them as needed. Consult the [Journal Article Reporting Standards](#) for what is needed for your type of article.

Participants

Who are they? How were they recruited? Report criteria for participant inclusion and exclusion. Perhaps some basic demographic stats are in order. A table is a great way to avoid repetition in statistical reporting.

Measures

This section can also be titled **Materials** or **Apparatus**. Whatever tools, equipment, or measurement devices used in the study should be described.

Measure A

Describe Measure A.

Measure B

Describe Measure B.

Subscale B1. A paragraph after a 4th-level header will appear on the same line as the header.

Subscale B2. A paragraph after a 4th-level header will appear on the same line as the header.

Subscale B2a. A paragraph after a 5th-level header will appear on the same line as the header.

Subscale B2b. A paragraph after a 5th-level header will appear on the same line as the header.

Procedure

What did participants do? How are the data going to be analyzed?

Results

Descriptive Statistics

Describe the basic characteristics of the primary variables. My ideal is to describe the variables well enough that someone conducting a meta-analysis can include the study without needing to ask for additional information.

Table 1 is an example of a plain markdown table. Note the that the caption begins with a colon.

Table 1

My Caption.

Letters	Numbers
A	1
B	2
C	3

Note. My note

Discussion

Describe results in non-statistical terms.

Limitations and Future Directions

Every study has limitations. Based on this study, some additional steps might include...

Conclusion

Describe the main point of the paper.

References

References marked with an asterisk indicate studies included in the meta-analysis.